

Mahatma Education Society's
Pillai HOC College of Engineering and Technology, Rasayani
Department of Computer Engineering

Subject: **Artificial Intelligence**

Lab Manual

Prepared By: Rahul Kapse

Mahatma Education Society's

Pillai HOC College of Engineering and Technology, Rasayani

Department of Computer Engineering

Practical List

Sr. No	Name of the Experiment	Page No.
1	One case study on AI applications published in IEEE/ACM/Springer or any prominent journal.	
2	Assignments on State space formulation and PEAS representation for various AI applications	
3	Program on uninformed search methods	
4	Program on informed search methods.	
5	Program on Game playing algorithms.	
6	Program for first order Logic	
7	Planning Programming	
8	Implementation for Bayes Belief Network	

H/W Requirements	RAM 512 MB, Printer, Cartridges
S/W Requirements	Python, Google COLAB, R studio

Experiment Mapping with lab Outcome

[illegible]

PRACTICAL NO: 01

Name :- _____

Roll No:- _____ Division:- _____ Batch:- _____

Date of Performance:- _____ Staff Signature with Date :- _____

Marks

--

TITLE: One case study on AI applications published in IEEE/ACM/Springer or any prominent journal.

Problem Definition: To make a detailed study of any Expert system or NLP application based on papers published in IEEE/ACM/Springer or any prominent journal.

Software used: Microsoft Office 2010

Theory

1. Form a group of 3-4.
2. Search for a paper.
3. Prepare a report of at least 4 pages containing the following,

Title of Paper:

Abstract of paper:

Procedure/Methodology:

References:

Conclusion

.....

.....

.....

.....

.....

LAB ASSIGNMENT

1) Define Artificial Intelligence

.....

.....

.....

.....

2) Identify the AI components in your case study?

.....

.....

.....

.....

3) Categorize AI into different 4 categories

.....

.....

.....

4) what are foundations of AI

.....

.....

.....

.....

5) What are applications of AI

.....

.....

.....

.....

PRACTICAL NO: 02

Name :- _____

Roll No:- _____ Division:- _____ Batch:- _____

Date of Performance:- _____ Staff Signature with Date : _____

Marks

--

TITLE: State Space Formulation and PEAS Representation**Problem Definition:** Assignments on State space formulation and PEAS representation for various AI applications**Software used:** Microsoft word**Theory:****PEAS representation:** In designing an agent, the first step must always be to specify the task environment as fully as possible.PEAS stands for **Performance measures, Environment, Actuators and Sensors**.

- **Performance measures:** These are the parameters used to measure the performance of the agent. How well the agent is carrying out a particular assigned task.
 - **Environment:** It is the task environment of the agent. The agent takes interacts with its environment. It takes perceptual input from the input and acts on the environment using actuators.
 - **Actuators:** These are the means of performing calculated actions on the environment. For a human agent; hands and legs are the actuators.
 - **Sensors:** These are the means of taking the input from the environment. For a human agent; ears, eyes and nose are the sensors.
-
- Use automated taxi driver as an example
 - Performance measure
 - How can we judge the automated driver?
 - Which factors are considered?
 - getting to the correct destination
 - minimizing fuel consumption
 - minimizing the trip time and/or cost
 - minimizing the violations of traffic laws
 - maximizing the safety and comfort, etc.
 - Environment

- A taxi must deal with a variety of roads
- Traffic lights, other vehicles, pedestrians, stray animals, road works, police cars, etc.
- Interact with the customer
- Actuators (for outputs)
 - Control over the accelerator, steering, gear shifting and braking
 - A display to communicate with the customers
- Sensors (for inputs)
 - Detect other vehicles, road situations
 - GPS (Global Positioning System) to know where the taxi is
 - Many more devices are necessary
- A sketch of automated taxi driver

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

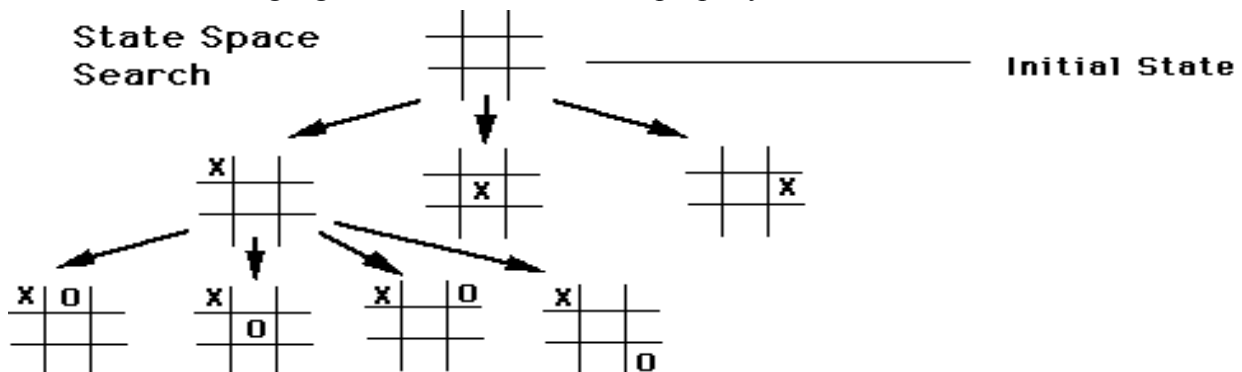
Figure 2.4 PEAS description of the task environment for an automated taxi.

- **Properties of task environments:** Fully observable vs. Partially observable
 - If an agent's sensors give it access to the complete state of the environment at each point in time then the environment is effectively and fully observable
 - if the sensors detect all aspects
 - That are relevant to the choice of action
- Partially observable
- An environment might be Partially observable because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data.
- Example:
 - A local dirt sensor of the cleaner cannot tell
 - Whether other squares are clean or not
- Deterministic vs. stochastic
 - next state of the environment Completely determined by the current state and the actions executed by the agent, then the environment is deterministic, otherwise, it is Stochastic.
 - Strategic environment: deterministic except for actions of other agents
 - -Cleaner and taxi driver are:
 - Stochastic because of some unobservable aspects à noise or unknown
- Static vs. dynamic
 - A dynamic environment is always changing over time
 - E.g., the number of people in the street
 - While static environment

- E.g., the destination
- Semi-dynamic
 - environment is not changed over time
 - but the agent's performance score does
- Discrete vs. continuous
 - If there are a limited number of distinct states, clearly defined percepts and actions, the environment is discrete
 - E.g., Chess game
 - Continuous: Taxi driving
- Single agent VS. multiagent
 - Playing a crossword puzzle – single agent
 - Chess playing – two agents
 - Competitive multi agent environment
 - Chess playing
 - Cooperative multiagent environment
 - Automated taxi driver
 - Avoiding collision
- Known vs. unknown
 - This distinction refers not to the environment itself but to the agent's (or designer's) state of knowledge about the environment.
 - In a known environment, the outcomes for all actions are given. (Example: solitaire card games).
 - If the environment is unknown, the agent will have to learn how it works in order to make good decisions. (Example: new video game).

State Space formulation:

State space search is a process used in the field of computer science, including artificial intelligence (AI), in which successive configurations or states of an instance are considered, with the intention of finding a goal state with the desired property.



Example: Representing Xs and Os as state-space problem.

State space representation of a problem:

1. All the states the system can be in are represented as *nodes* of a graph.
2. An action that can change the system from one state to another (e.g. a move in a game) is represented by a *link* from one node to another.
3. Links may be *unidirectional* (e.g. Xs and Os, chess, can't go back) or *bi-directional* (e.g. geographic move).
4. Search for a solution.
5. A solution might be:
 1. *Any* path from start state to goal state.
 2. The *best* (e.g. lowest cost) path from start state to goal state (e.g. **Travelling salesman problem**).
6. It may be possible to reach the same state through many *different* paths (obviously true in Xs and Os).
7. There may be *loops* in the graph (can go round in circle). No loops in Xs and Os.

Conclusion

.....

.....

.....

.....

.....

.....

LAB ASSIGNMENT

1) What are different types of task environments?

.....

.....

.....

.....

.....

2) Explain PEAS for playing tennis against wall

.....

.....

.....

.....

.....

3.Explain state space formulation for 8 tile problem

.....

.....

.....

.....

.....

PRACTICAL NO: 03

Name :- _____

Roll No:- _____ Division:- _____ Batch:- _____

Date of Performance:- _____ Staff Signature with Date : _____

Marks

--

TITLE: Program on uninformed search methods.**Aim : Implement any un-inform search algorithm****Theory:**

Uninformed strategies use only the information available in the problem definition also known as blind searching. The search algorithm produces the search tree without using any domain knowledge, which is a brute force in nature. They don't have any background information on how to approach the goal or whatsoever. But these are the basics of search algorithms in AI.

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

Terms which will be frequently used:

- **State:** It provides all the information about the environment.
- **Goal State:** The desired resulting condition in a given problem and the kind of search algorithm we are looking for.
- **Goal Test:** The test to determine whether a particular state is a goal state.
- **Path/Step Cost:** These are integers that represent the cost to move from one node to another node.
- **Space Complexity:** A function describing the amount of space(memory) an algorithm takes in terms of input to the algorithm.
- **Time Complexity:** A function describing the amount of time the algorithm takes in terms of input to the algorithm.
- **Optimal:** Extent of preference of the algorithm
- **'b** – maximum branching factor in a tree.

- ‘**d** – the depth of the least-cost solution.
- ‘**m** – maximum depth state space (maybe infinity)

1. Breadth-first search

- Recall from Data Structures the basic algorithm for a breadth-first search on a graph or tree
- Expand the *shallowest (little depth)* unexpanded node
- Place all new successors at the end of a FIFO queue

2. Uniform-Cost Search

- Same idea as the algorithm for breadth-first search...but...
 - Expand the least-cost unexpanded node
- FIFO queue is ordered by cost
- Equivalent to regular breadth-first search if all step costs are equal

3. Depth-First Search

- Recall from Data Structures the basic algorithm for a depth-first search on a graph or tree
- Expand the deepest unexpanded node (current fringe of the search tree)
- Unexplored successors are placed on a stack until fully explored

4. Depth-Limited Search

- A variation of depth-first search that uses a depth limit
 - Alleviates the problem of unbounded trees
- Search to a predetermined depth l (“ell”)
- Nodes at depth l have no successors
- Same as depth-first search if $l = \infty$
- Can terminate for failure and cutoff

5. Iterative Deepening Search

- Iterative deepening depth-first search
 - Uses depth-first search
 - Finds the best depth limit
- Gradually increases the depth limit; 0, 1, 2, ... until a goal is found

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: ^a complete if b is finite; ^b complete if step costs $\geq \epsilon$ for positive ϵ ; ^c optimal if step costs are all identical; ^d if both directions use breadth-first search.

Conclusion

.....

.....

.....

LAB ASSIGNMENT

1) Explain Problem solving agent?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

2) Describe evolution tree-search strategies of uninformed search algorithms.

.....

.....

[illegible]

3.Explain iterative deepening algorithm.

[illegible]

PRACTICAL NO: 04

Name :- _____

Roll No:- _____ Division:- _____ Batch:- _____

Date of Performance:- _____ Staff Signature with Date : _____

Marks

TITLE: Program on informed search method**Aim: Implement A* Algorithm****Theory: Informed Search**

- A search using domain-specific knowledge.
- Suppose that we have a way to estimate how close a state is to the goal, with an evaluation function.
- General strategy: expand the best state in the open list first. It's called a best-first search or ordered state-space search.
- In general the evaluation function is imprecise, which makes the method a heuristic (works well in most cases).
- The evaluation is often based on empirical observations.

Examples

- The vacuum cleaner - it might move from a cell towards the dirtiest adjacent cell.
- For a path-search in a graph with a geometrical representation - give preference to the neighbours which are the closest to the target based on the Euclidian distance (which may or may not be an indication of a good path). This is called a greedy search algorithm.
- For a game-playing, the algorithm may move to a state giving it a strategic advantage (capturing the opponent's queen).

Heuristic Examples

- Tic-Tac-Toe: the number of tuples (2 or 3) or pieces that are aligned.
- Jigsaw puzzle: the length of the completed border and the number of large continuous completed areas.

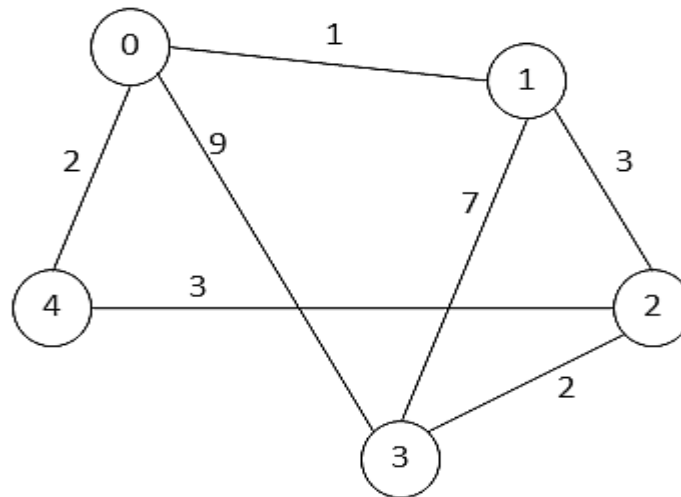
- The 8-Queens problem: the number of cells that are not yet attacked by any of the queens.
- For a maze: the Manhattan distance to the target
row difference + column difference.

The A* Search

- A well-known method that minimizes the estimated solution cost.
- Let $g(n)$ be the cost of the path from the initial state to the current state n .
- Let $h(n)$ be the estimated distance from the current state to the goal. This is a heuristic.
- The estimated cost of the cheapest solution through n is
 $f(n) = g(n) + h(n)$
- Given a good heuristic h , A* is both complete and optimal.
- The method is similar to the best-first where "best" is defined by the estimated cost $f(n)$.

```
def A_star(initial):
    open_list = [initial]
    close_list = []
    goal = False
    while open_list and not goal:
        state = pop(open_list)
        if state in close_list: continue
        else: close_list.append(state)
        if (reach_goal(state)):
            goal = state
        else:
            children = expand-state(state)
            open_list += children
            open_list.sort(f_value)
    return goal
```


Small Example of Graph



Artific

Admissible Heuristic

- A heuristic h is called admissible if it never overestimates the cost to reach the goal from a state. Which means that $h(n) < \text{Min}(\text{cost}(\text{path}(n, \text{goal})))$ over all paths leading from n to the goal.
- Search for a path in a geometric graph: the length of a path from vertex A to vertex B can never be less than the distance between A and B . So the actual distance is an admissible heuristic.

Consistent Heuristic

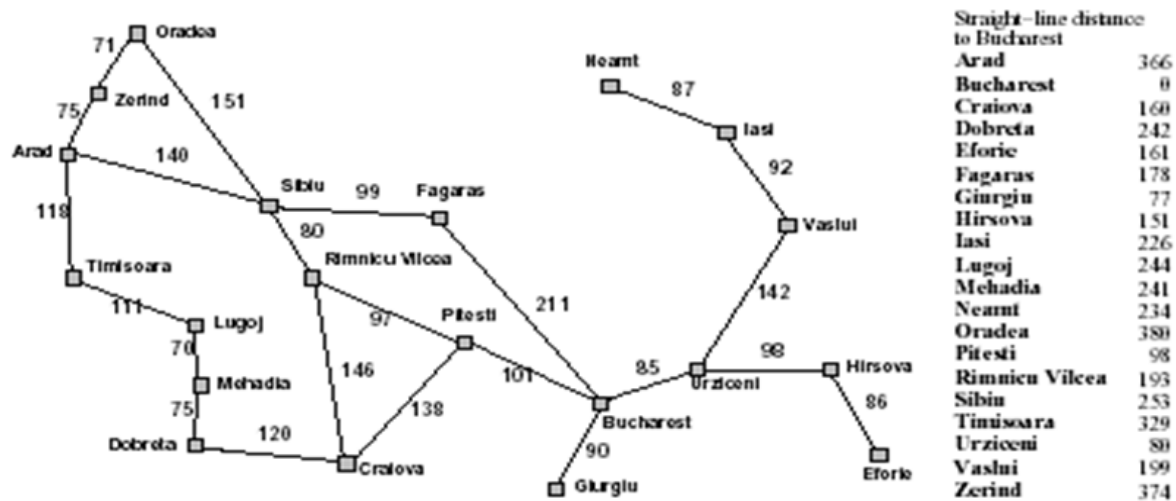
- A heuristic is consistent if for any n and c where c is a child state of n , $h(n) < h(c) + \text{cost}(\text{edge}(n, c))$
- This is known as the triangular inequality.

Optimality and Completeness

- A^* with an admissible heuristic is optimal for a tree search, meaning when there is only one path to any given state.
- A^* with a consistent heuristic is optimal for any kind of search (graph search), even if several paths can lead to the same state. In this case we must not exclude states we have seen before, but update their information when their information when the new path is better.
- A^* is complete if the search graph is finite or when there is no infinite path in the graph having a finite total cost.

Example

Romania with step costs in km



Starting from: timisoara. Goal: bucharest.

lugoj: $g = 111$, $h = 244$, $f = 355$
 ((lugoj 355) (arad, 484))
 mehadia: $g = 181$, $h = 241$, $f = 422$
 ((mehadia 422) (arad 484))
 ((arad 484) (dobreta 498))
 ((dobreta 498) (sibiu 511) (zerind 567))
 ((sibiu 511) (craiova 525) (zerind 567))
 ((craiova 525) (rimnicu 531) (fagaras 535) (zerind 567))
 ((rimnicu 531) (fagaras 535) (zerind 567) (pitesti 611))
 ((pitesti 533) (fagaras 535) (zerind 567))
 ((fagaras 535) (bucharest 546) ((zerind 567))
 ((bucharest 546) (zerind 567) (bucharest 568))

Path: timisoara->arad->sibiu->rimnicu->pitesti->bucharest

```
def A_star(initial):
    open_list = [initial]
    close_list = []
    goal = False
    while open_list and not goal:
        state = pop(open_list)
        if state in close_list: continue
```

```
    else: close_list.append(state)
    if (reach_goal(state)):
        goal = state
    else:
        children = expand-state(state)
        open_list += children
        open_list.sort(f_value)
return goal
```

Conclusion

.....

.....

.....

.....

.....

.....

LAB ASSIGNMENT

Q1. Briefly describe what is uninformed and informed search? Name examples of uninformed and informed search strategies in lists and trees.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Q2. What is the heuristic function of greedy best-first search?

.....

.....

.....

.....

.....

Q3. Explain admissible heuristics in 8 puzzle problem.

This image shows a full page of white paper with ten horizontal dashed lines, typical of primary-ruled notebook paper. The lines are evenly spaced and extend across the entire width of the page. There are no margins, text, or other markings present.

Q4. Explain properties of A* Algorithms..

.....

.....

.....

[illegible]

Q5. Explain in short memory bounded heuristic search.

[illegible]

PRACTICAL NO: 05

Name :- _____

Roll No:- _____ Division:- _____ Batch:- _____

Date of Performance:- _____ Staff Signature with Date : _____

Marks

TITLE: Alpha Beta Pruning

Problem Definition: To implement a program to show alpha beta pruning for Tic-Tac-Toe Problem.

Software used: python

Theory

The problem with min-max search is that the number of game states it has to examine is exponential in the depth of the tree. Unfortunately, we can't eliminate the exponent, but it turns out we can effectively cut it in half. The trick is that it is possible to compute the correct min-max decision without looking at every node in the game tree. That is, we can borrow the idea of **pruning** to eliminate large parts of the tree from consideration. The particular technique we examine is called **alpha beta pruning**. When applied to a standard min-max tree, it returns the same move as min-max would, but prunes away branches that cannot possibly influence the final decision.

Consider again the two-ply game tree from Figure 8.1

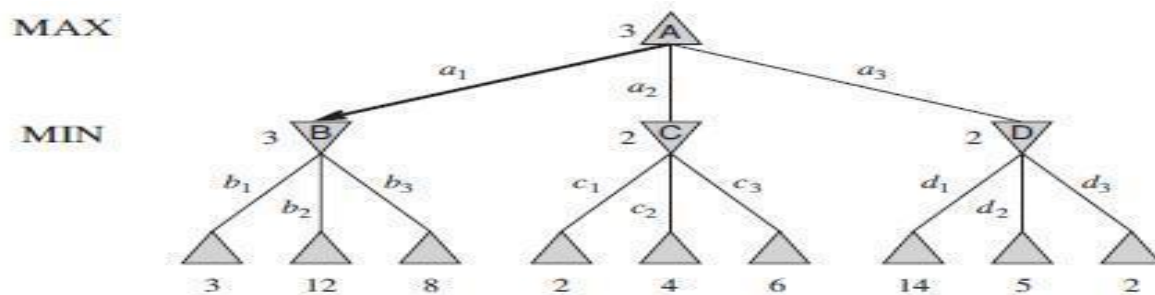


Figure 8.1: A two-ply game tree. The nodes are “MAX nodes,” in which it is MAX’s

turn to move, and the nodes are “MIN nodes.” The terminal nodes show the utility values for MAX; the other nodes are labelled with their min-max values. MAX’s best move at the root is a1, because it leads to the state with the highest min-max value, and MIN’s best reply is b1, because it leads to the state with the lowest min-max value.

The calculation of the optimal decision at each point in the process. The steps are explained in Figure 8.2. The outcome is that we can identify the min-max decision without ever evaluating two of the leaf nodes.

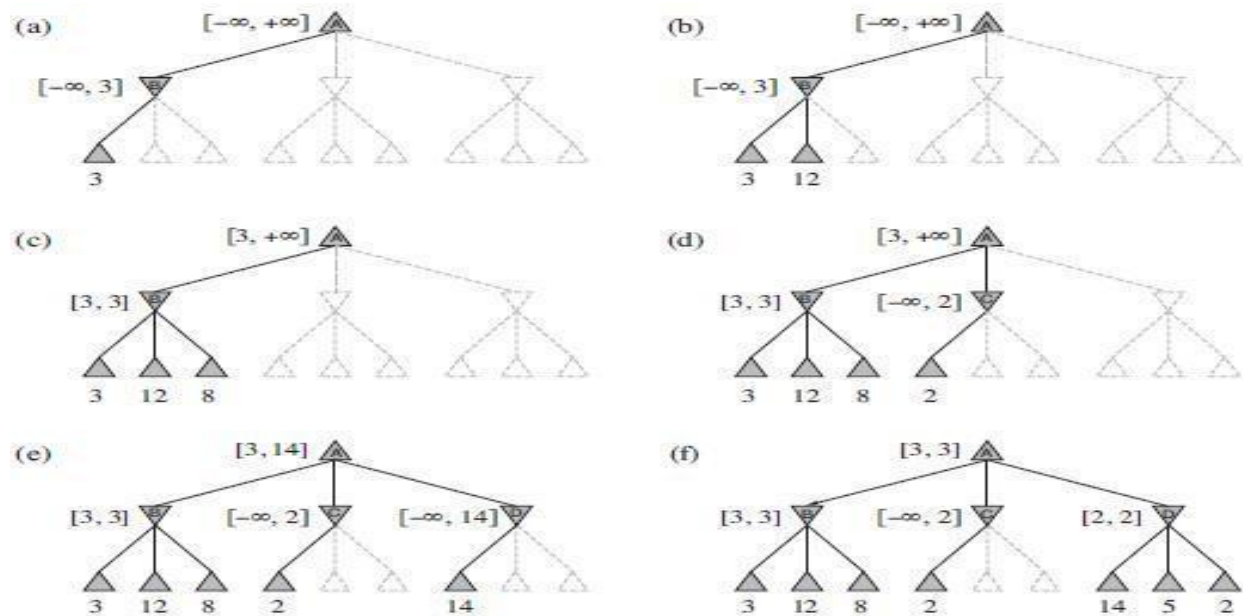


Figure 8.2 Stages in the calculation of the optimal decision for the game tree in Figure 5.2. At each point, we show the range of possible values for each node. (a) The first leaf below B has the value 3. Hence, B, which is a MIN node, has a value of *at most* 3. (b) The second leaf below B has a value of 12; MIN would avoid this move, so the value of B is still *at most* 3. The third leaf below B has a value of 8; we have seen all B's successor states, so the value of B is exactly 3. Now, we can infer that the value of the root is *at least* 3, because MAX has a choice worth 3 at the root. (d) The first leaf below C has the value 2. Hence, C, which is a MIN node, has a value of *at most* 2. But we know that B is worth 3, so MAX would never choose C. Therefore, there is no point in looking at the other successor states of C. This is an example of alpha-beta pruning. (e) The first leaf below D has the value 14, so D is worth *at most* 14. This is still higher than MAX's best alternative (i.e., 3), so we need to keep exploring D's successor states. Notice also that we now have bounds on all of the successors of the root, so the root's value is also *at most* 14. (f) The second successor of D is worth 5, so again we need to keep exploring. The third successor is worth 2, so now D is worth exactly 2. MAX's decision at the root is to move to B, giving a value of 3.

Another way to look at this is as a simplification of the formula for MINIMAX. Let the two unevaluated successors of node C in Figure 8.2 have values x and y . Then the value of the root node is given by

$$\text{MINIMAX}(\text{root}) = \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2))$$

- $\max(3, \min(2, x, y), 2)$
- $\max(3, z, 2)$ where $z = \min(2, x, y) \leq 2$
- 3.

In other words, the value of the root and hence the min-max decision are *independent* of the values of the pruned leaves x and y .

Alpha–beta pruning can be applied to trees of any depth, and it is often possible to prune entire sub-trees rather than just leaves. The general principle is this: consider a node n somewhere in the tree such that Player has a choice of moving to that node.

If Player has a better choice m either at the parent node of n or at any choice point further up, then n *will never be reached in actual play*. So once we have found out enough about n (by examining some of its descendants) to reach this conclusion, we can prune it.

Min-max search is depth-first, so at any one time we just have to consider the nodes along a single path in the tree. Alpha–beta pruning gets its name from the following two parameters that describe bounds on the backed-up values that appear anywhere along the path:

- α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.
- β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.

Alpha–beta search updates the values of α and β as it goes along and prunes the remaining

branches at a node (i.e., terminates the recursive call) as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.

Algorithm

function ALPHA-BETA-SEARCH(state) **returns** an action v
 $\leftarrow$ MAX-VALUE(state, $-\infty$, $+\infty$)

return the action in ACTIONS(state) with value v

function MAX-VALUE(state, α , β) **returns** a utility value **if**
 TERMINAL-TEST(state) **then return** UTILITY(state) $v \leftarrow$
 $-\infty$

for each a **in** ACTIONS(state) **do**

$v \leftarrow$ MAX(v , MINVALUE(RESULT(s, a), α , β)) **if** $v \geq$
 β **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$
return v

function MIN-VALUE($\text{state}, \alpha, \beta$) **returns** a utility value **if**
 TERMINAL-TEST(state) **then return** UTILITY(state)

$v \leftarrow +\infty$

for each a **in** ACTIONS(state) **do**

$v \leftarrow \text{MIN}(v, \text{MAXVALUE}(\text{RESULT}(s, a), \alpha, \beta))$

if $v \leq \alpha$ **then return** v

$\beta \leftarrow \text{MIN}(\beta, v)$
return v

Conclusion

.....

.....

.....

.....

.....

.....

LAB ASSIGNMENT

1. What do you mean by adversarial search?

.....

.....

.....

.....

.....

2. What is a game tree?

.....

.....

.....

.....

.....

3. Why do we require pruning in a game tree?

.....

.....

.....

.....

.....

4. What is the complexity of alpha beta pruning algorithm?

.....

.....

.....

.....

.....

5. For the tree in figure 8.1 calculate the minmax value

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

6. For the tree in figure 8.1 show the winning path.

.....

.....

.....

.....

.....

.....

.....

.....

.....

7. Draw a partial game tree for tic-tac-toe game and show associated utility at each step.

.....

.....

.....

.....

.....

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the entire width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the paper.

PRACTICAL NO: 6

Name of the Student:- _____

Roll No. _____ Subject:- _____

Date of Practical Performed:- _____ Staff Signature with Date

Marks

TITLE: Introduce AI programming Language (PROLOG programming)

AIM: Implement a prolog program to demonstrate a family tree

Theory: There are only three basic constructs in Prolog: facts, rules, and queries. A collection of facts and rules is called a knowledge base (or a database) and Prolog programming is all about writing knowledge bases. That is, Prolog programs simply are knowledge bases, collections of facts and rules which describe some collection of relationships that we find interesting. So how do we use a Prolog program? By posing queries. That is, by asking questions about the information stored in the knowledge base.

Introduction to SWI- PROLOG

Swi-Prolog can be freely downloaded from <http://www.swi-prolog.org> and can easily install it.

Run SWI-PROLOG

Click: start -> Program files -> SWI-PROLOG Prolog

Create a source file

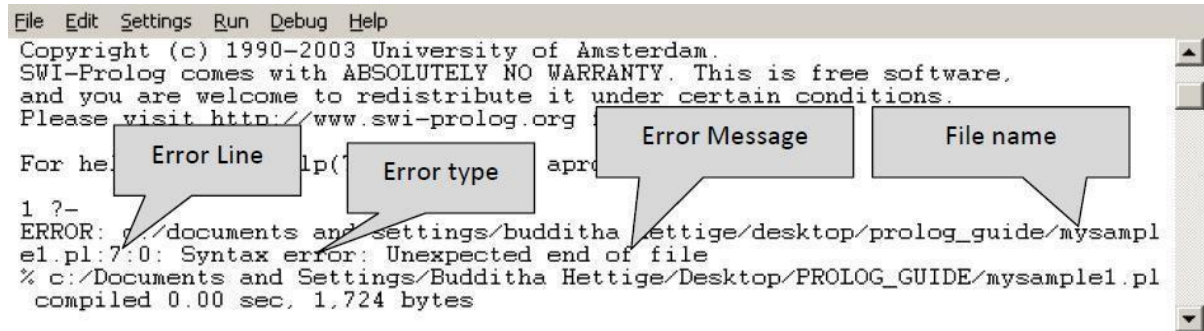
1. Open Text pad Using Windows start menu
2. Enter some text in your text document
3. Save your source file as a .pl extension (mysample1.pl)

Consult a source file

1. In a prolog window click file menu
2. Click consult
3. Select appropriate file and press the open button.

Update your source file and Re-consult

1. File -> consult -> select mysample1.pl file -> open
2. Type consult (test.pl). in the prompt

PROLOG help***Errors*****Figure 2.1: Compilation errors in prolog*****Create a Simple facts in a knowledge base***

The Knowledge Base is simply a collection of facts. Facts are used to state things that are unconditionally true of the domain of interest. For example, we can state that Saman, ruwan and kamal are students, and kamal play cricket, using the following student(saman).

```
woman(ruwan).
woman(kamal).
playcricket(kamal).
```

In general a fact has a name and values. The general format is

```
fact_name (data1, data2, ... ).
Student(_Saman Kumara').
Student(_Saman kumara', _34/1, Colombo 3', _as1003').
course(_csu2280', _Deductive Reasoning and PROLOG for AI').
```

Create a Prolog Facts to store student name, age and sex.

```
student(saman,12,male).
```

Facts are used to state things that are unconditionally true of some situation of interest. For example, we can state that Mia, Jody, and Yolanda are women, that Jody plays air guitar, and that a party is taking place, using the following five facts:

```
woman(mia).
woman(jody).
woman(yolanda).
playsAirGuitar(jody).
party.
```

This collection of facts is KB1. It is our first example of a Prolog program. Note that the names mia , jody , and yolanda , the properties woman and playsAirGuitar , and the proposition party have been written so that the first letter is in lower-case. This is important; we will see why a little later on.

How can we use KB1? By posing queries. That is, by asking questions about the information KB1 contains. Here are some examples. We can ask Prolog whether Mia is a woman by posing the query:

```
?- woman(mia).
```

Prolog will answer

```
yes
```

for the obvious reason that this is one of the facts explicitly recorded in KB1. Incidentally, we don't type in the `?- .` This symbol (or something like it, depending on the implementation of Prolog you are using) is the prompt symbol that the Prolog interpreter displays when it is waiting to evaluate a query. We just type in the actual query (for example `woman(mia)`) followed by `.` (a full stop). The full stop is important. If you don't type it, Prolog won't start working on the query.

Starting Prolog and loading a program.

You can start it by clicking the Prolog icon. As a result, a SWI-Prolog console will open

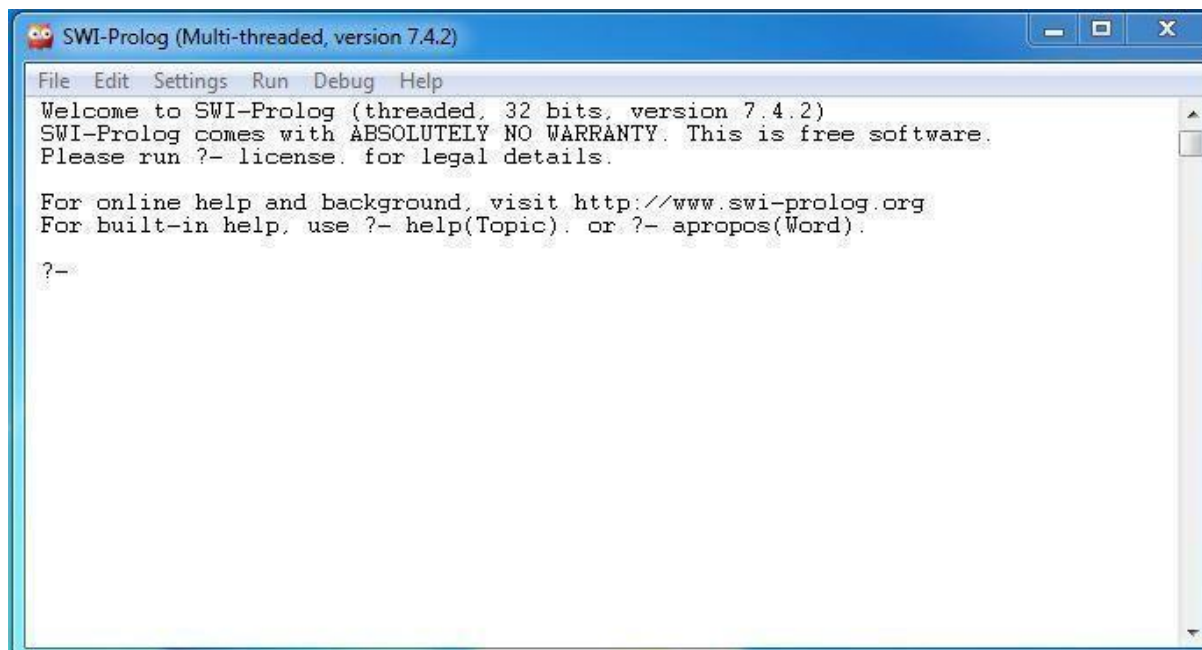


Figure 2.2: SWI-Prolog console

Alternatively, you can start Prolog by the SWI-Prolog executable plwin.exe (default folder C:\Program Files\pl\bin) or by opening a .pl file holding Prolog program text. For example, you can practice using the file likes.pl available in C:\Program Files\pl\demo. Another way to open likes.pl is demonstrated in Figure below:

```
?- [swi('demo/likes')].
% swi('demo/likes') compiled 0.06 sec, 2,420 bytes
```

```
Yes
```

```
?-
```

On

Windows, the .pl extension is associated with swipl-win.exe and most comfortable way is to **double-click** the .pl file you want to load in the explorer. This will start SWI-Prolog, which changes directory to the directory holding the file and then loads the clicked file.

Basic Prolog Commands

A few essential commands are:

1. halt.

exit Prolog (short form: control-d)

2. consult(filename).

load the file named filename.pl (note the added .pl extension). If you need some more complex name (for example with a path), put it in single quotes, for example consult('/Users/schmertz Kopf/squid.pl').

3. [filename]. shorthand for consult

4. help(topic) or apropos(topic).

Brings up the relevant section of the manual in a separate window (so requires X for linux / Mac).

Menu commands

The SWI-Prolog console has a menu for accessing the most commonly used commands (see the first line in Figure 2.1). In particular, if you click on the button File of the menu you will see the following entries:

Consult, Edit, New, Reload modified files, Navigator, Exit.

The command Reload modified files is to reload all loaded source-files that have been modified using the make command .

The command Navigator opens an explorer-like view on Prolog files and the predicates they contain: to view the predicates in a file you should find the file by Navigator and then click on the file name. Also Navigator provides an option to edit the selected file.

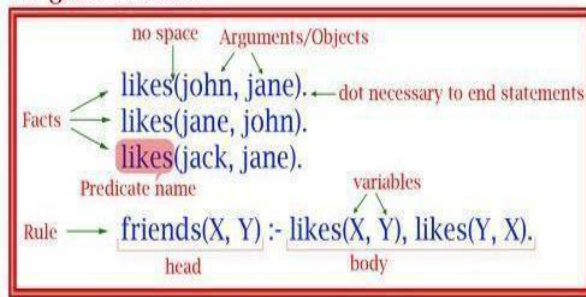
The menu button Settings allows you to change the font of the console as well as providing some additional options.

The button Run provides two options: Interrupt and New thread. Interrupt is to interrupt the running Prolog process. New thread creates a new window running in a separate thread of execution.

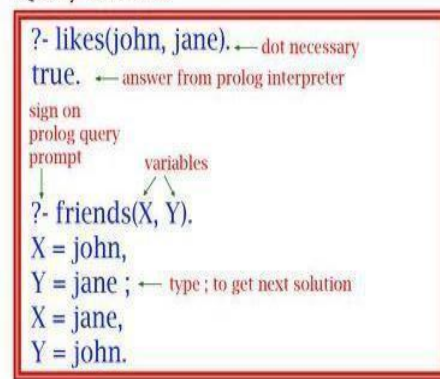
The button Help provides access to the online Prolog manual. In particular, it contains links to pages of the SWI-Prolog website.

Prolog always performs depth-first-search, Matches facts & rules (i.e. knowledge base) in top-down manner and resolves the goals or sub goals in left-to-right manner. Most Important thing to keep in mind while writing prolog program - "order of writing facts & rules always matters".

Program Window



Query Window



PROGRAM

Consider the following Family Tree

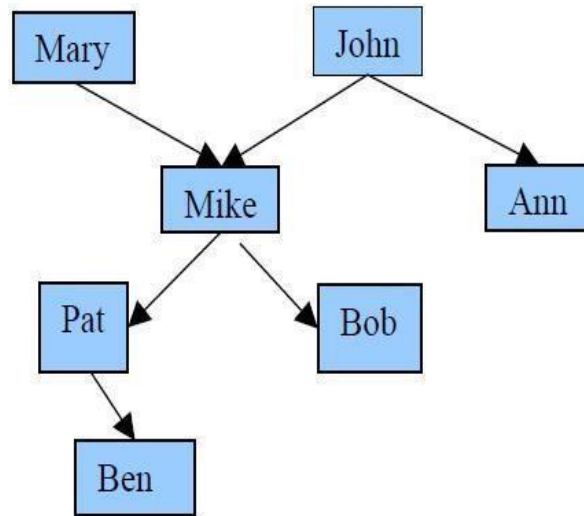


Figure 2.4: Sample Family tree

To write a program which defines a family relation click the button File of the menu and choose the option New and name it as family.pl. Type the text in the file and save it

```
parent(john, ann). /*the fact that John is a parent of Ann*/
```

```
parent(john, mike). %the fact that John is a parent of Mike
```

```
parent(mary, mike).
```

```
parent(mike, pat).
```

```
parent(mike, bob).
```

```
parent(pat, ben).
```

Here parent is the name of a family relation. The text in /* */ is a comment as well as the text between the percent character % and the end of the line (see the first and, respectively, the second lines above).

Consult the file, use the command

```
?- consult(family).
```

Now we can pose Prolog questions about the relation parent

```

?- parent(john, mike).      /* Is John a parent of Mike? */
Yes
?- parent(mike, ann).      /* Is Mike a parent of Ann? */
No
?- parent(mike, X).        /* Who is a child of Mike? */
X = pat
Yes
?- parent(mike, _).        /* Is Mike a parent? */
Yes

```

If we want to get a list of all pairs parent-child, type parent(X, Y), and then press repeatedly semicolon (;) . To abort the process, press return.

```

?- parent(X, Y).
X = john
Y = ann ;

X = john
Y = mike ;

X = mary
Y = mike ;

X = mike
Y = pat ;

X = mike
Y = bob ;

X = pat
Y = ben ;
No                                     /*indicates that the list is full*/

```

To extend the program by adding new facts or rules, edit the file family.pl. You can use, for example, option File/Edit of the menu, which will open the file family.pl for editing. For example, add information on the gender and define predicates mother, father

```

parent(john, ann). %the fact that John is a parent of Ann
parent(john, mike). %the fact that John is a parent of Mike
parent(mary, mike).
parent(mike, pat).
parent(mike, bob).
parent(pat, ben).
female(mary). %the fact that Mary is a female
female(ann).
female(pat).
male(john). %the fact that John is a male
male(mike).
male(bob).

```

```
male(ben).
mother(X,Y) :- parent(X, Y), female(X). %definition of the predicate mother
father(X,Y) :- parent(X, Y), male(X). %definition of the predicate father
Consult the file smith.pl. Now you can ask new questions. For example, try the following:
?- female(X).
?- mother(X, mike).
?- mother(pat, Y).
?- father(X, Y).
```

Conclusion

.....
.....
.....

LAB ASSIGNMENT

1) What are rules in prolog? Give example

.....
.....
.....
.....
.....

2) What are facts in prolog? Give example

.....
.....
.....
.....

3) What is the command to load a file in prolog?

.....
.....

4) How do we define goals in prolog?

.....
.....
.....
.....

5) For given program how would prolog respond to the query if you keep entering ';' after each solution?

Program :
p(a,b). p(b,c).
p(X, Y):-p(Y, X).
Query : ?- p(X,Y).

PRACTICAL NO: 7

Name of the Student:- _____

Roll No. _____ Subject:- _____

Date of Practical Performed:- _____ Staff Signature with Date

Marks

TITLE: Planning Programming

AIM: Implement a Blocks-World planning problem

Theory: AI Planning is a field of Artificial Intelligence which explores the process of using autonomous techniques to solve planning and scheduling problems. A planning problem is one in which we have some initial starting state, which we wish to transform into a desired goal state through the application of a set of actions.

Planning and its types

This activity of coming up with a sequence of actions in order to accomplish the target or goal is called Planning. Planning can be Classical or Non-classical. In case of Classical Planning, the environment is fully observable, deterministic, static and discrete, whereas in case of Non-classical Planning, the environment is partially observable (i.e. the entire state of the environment is not visible at a given instant) or non- deterministic (or stochastic, i.e. the current state and chosen action cannot completely determine the next state of the environment).

Planning languages

Representation of planning problems is often done using STRIPS (Stanford Research Institute Problem Solver). It consists of:

A set of states- It is a conjunction of positive ground literals.

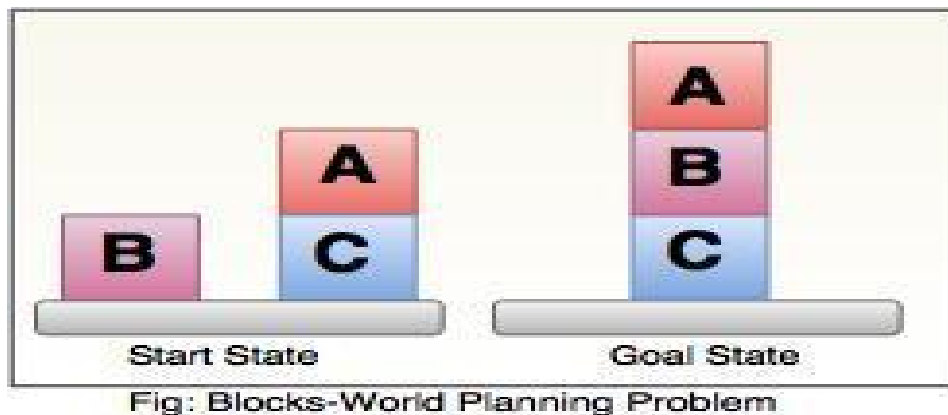
A set of goals- partially specified state represented as a conjunction of positive literals, and

A set of actions- For each action, there is a precondition that must be satisfied and an effect which reflects the impact the action has on the environment after it has been performed.

Planning problems can also be represented using PDDL (Planning Domain Definition Language).

Blocks-World planning problem

- The blocks-world problem is known as **Sussman Anomaly**.
- Non Interleaved planners of the early 1970s were unable to solve this problem, hence it is considered as anomalous.
- When two subgoals G1 and G2 are given, a non interleaved planner produces either a plan for G1 concatenated with a plan for G2, or vice-versa.
- In the blocks-world problem, three blocks labeled as 'A', 'B', 'C' are allowed to rest on the flat surface. The given condition is that only one block can be moved at a time to achieve the goal.
- The start state and goal state are shown in the following diagram.



Components of Planning System

The planning consists of following important steps:

- Choose the best rule for applying the next rule based on the best available heuristics.
- Apply the chosen rule for computing the new problem state.
- Detect when a solution has been found.
- Detect dead ends so that they can be abandoned and the system's effort is directed in more fruitful directions.
- Detect when an almost correct solution has been found.

Goal stack planning

This is one of the most important planning algorithms, which is specifically used by **STRIPS**.

- The stack is used in an algorithm to hold the action and satisfy the goal. A knowledge base is used to hold the current state, actions.
- Goal stack is similar to a node in a search tree, where the branches are created if there is a choice of an action.

The important steps of the algorithm are as stated below:

- i. Start by pushing the original goal on the stack. Repeat this until the stack becomes empty. If stack top is a compound goal, then push its unsatisfied subgoals on the stack.
- ii. If stack top is a single unsatisfied goal then, replace it by an action and push the action's precondition on the stack to satisfy the condition.
- iii. If stack top is an action, pop it from the stack, execute it and change the knowledge base by the effects of the action.
- iv. If stack top is a satisfied goal, pop it from the stack.

Non-linear planning

This planning is used to set a goal stack and is included in the search space of all possible subgoal orderings. It handles the goal interactions by interleaving method.

Advantage of non-Linear planning

Non-linear planning may be an optimal solution with respect to plan length (depending on search strategy used).

Disadvantages of Nonlinear planning

- It takes larger search space, since all possible goal orderings are taken into consideration.
- Complex algorithm to understand.

Algorithm

1. Choose a goal 'g' from the goalset
2. If 'g' does not match the state, then
 - Choose an operator 'o' whose add-list matches goal g
 - Push 'o' on the opstack

- Add the preconditions of 'o' to the goalset

3. While all preconditions of operator on top of opstack are met in state

- Pop operator o from top of opstack
- $state = apply(o, state)$
- $plan = [plan; o]$

Conclusion

.....

.....

.....

LAB ASSIGNMENT

1) What is planning in AI?

.....

.....

.....

.....

2) What are different types of planning?

.....

.....

.....

3) Explain Partial order planning in detail.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

4)Describe PDDL in brief.

.....

.....

.....

.....

Q5. Describe hierarchical planning in brief.

.....

.....

.....

.....

.....

.....

PRACTICAL NO: 8

Name of the Student:-_____

Roll No._____ Subject:-_____

Date of Practical Performed:-_____ Staff Signature with Date

Marks

TITLE: Bayes belief Network

AIM: Implement a Bayes belief Network

Theory:

A Bayesian Belief Network, or simply “*Bayesian Network*,” provides a simple way of applying Bayes Theorem to complex problems.

The networks are not exactly Bayesian by definition, although given that both the probability distributions for the random variables (nodes) and the relationships between the random variables (edges) are specified subjectively, the model can be thought to capture the “*belief*” about a complex domain.

Bayesian probability is the study of subjective probabilities or belief in an outcome, compared to the frequentist approach where probabilities are based purely on the past occurrence of the event.

A Bayesian Network captures the joint probabilities of the events represented by the model.

A Bayesian belief network describes the joint probability distribution for a set of variables.

Central to the Bayesian network is the notion of conditional independence.

Independence refers to a random variable that is unaffected by all other variables. A dependent variable is a random variable whose probability is conditional on one or more other random variables.

Conditional independence describes the relationship among multiple random variables, where a given variable may be conditionally independent of one or more other random variables. This does not mean that the variable is independent per se; instead, it is a clear definition that the variable is independent of specific other known random variables.

A probabilistic graphical model, such as a Bayesian Network, provides a way of defining a probabilistic model for a complex problem by stating all of the conditional independence assumptions for the known variables, whilst allowing the presence of unknown (latent) variables.

As such, both the presence and the absence of edges in the graphical model are important in the interpretation of the model.

Bayesian networks provide useful benefits as a probabilistic model.

For example:

- **Visualization.** The model provides a direct way to visualize the structure of the model and motivate the design of new models.
- **Relationships.** Provides insights into the presence and absence of the relationships between random variables.
- **Computations.** Provides a way to structure complex probability calculations.

How to Develop and Use a Bayesian Network

Designing a Bayesian Network requires defining at least three things:

- **Random Variables.** What are the random variables in the problem?
- **Conditional Relationships.** What are the conditional relationships between the variables?
- **Probability Distributions.** What are the probability distributions for each variable?

It may be possible for an expert in the problem domain to specify some or all of these aspects in the design of the model.

In many cases, the architecture or topology of the graphical model can be specified by an expert, but the probability distributions must be estimated from data from the domain.

Both the probability distributions and the graph structure itself can be estimated from data, although it can be a challenging process. As such, it is common to use learning algorithms for this purpose; for example, assuming a Gaussian distribution for continuous random variables gradient ascent for estimating the distribution parameters.

Once a Bayesian Network has been prepared for a domain, it can be used for reasoning, e.g. making decisions.

Reasoning is achieved via inference with the model for a given situation. For example, the outcome for some events is known and plugged into the random variables. The model can be used to estimate the probability of causes for the events or possible further outcomes.

Reasoning (inference) is then performed by introducing evidence that sets variables in known states, and subsequently computing probabilities of interest, conditioned on this evidence.

Practical examples of using Bayesian Networks in practice include medicine (symptoms and diseases), bioinformatics (traits and genes), and speech recognition (utterances and time).

Example of a Bayesian Network

We can make Bayesian Networks concrete with a small example.

Consider a problem with three random variables: A, B, and C. A is dependent upon B, and C is dependent upon B.

We can state the conditional dependencies as follows:

- A is conditionally dependent upon B, e.g. $P(A|B)$
- C is conditionally dependent upon B, e.g. $P(C|B)$

We know that C and A have no effect on each other.

We can also state the conditional independencies as follows:

- A is conditionally independent from C: $P(A|B, C)$
- C is conditionally independent from A: $P(C|B, A)$

Notice that the conditional dependence is stated in the presence of the conditional independence. That is, A is conditionally independent of C, or A is conditionally dependent upon B in the presence of C.

We might also state the conditional independence of A given C as the conditional dependence of A given B, as A is unaffected by C and can be calculated from A given B alone.

- $P(A|C, B) = P(A|B)$

We can see that B is unaffected by A and C and has no parents; we can simply state the conditional independence of B from A and C as $P(B, P(A|B), P(C|B))$ or $P(B)$.

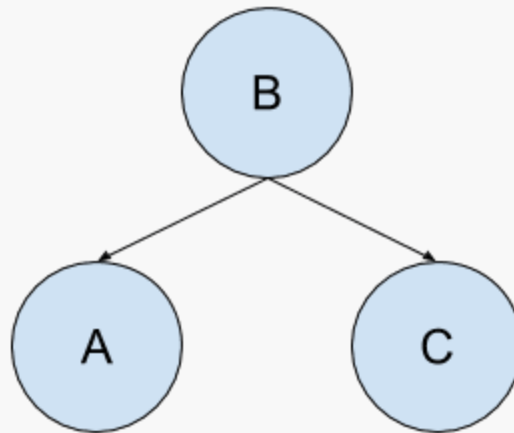
We can also write the joint probability of A and C given B or conditioned on B as the product of two conditional probabilities; for example:

- $P(A, C | B) = P(A|B) * P(C|B)$

The model summarizes the joint probability of $P(A, B, C)$, calculated as:

- $P(A, B, C) = P(A|B) * P(C|B) * P(B)$

We can draw the graph as follows:



Example of a Simple Bayesian Network

Notice that the random variables are each assigned a node, and the conditional probabilities are stated as directed connections between the nodes. Also notice that it is not possible to navigate the graph in a cycle, e.g. no loops are possible when navigating from node to node via the edges.

Also notice that the graph is useful even at this point where we don't know the probability distributions for the variables.

You might want to extend this example by using contrived probabilities for discrete events for each random variable and practice some simple inference for different scenarios.

Conclusion

.....
.....
.....

LAB ASSIGNMENT

1) What is the Bayesian belief network?

.....

.....

.....

.....

.....

2) Where are Bayesian belief networks used??

.....

.....

.....

.....

3) What are the advantages of the Bayesian belief network?

[illegible]

.....

.....

4)What are Bayesian networks? How are they useful?

.....

.....

.....

.....

Q5.What is the Bayesian network in AI explain it with a proper example?

.....

.....

.....

.....

.....

.....

.....

.....